

CCF 355: UFV Wilds - Parte 04

Emanuel Vitor Carvalho Ruella (3891)

Marcus Vinicius Guerra Ribeiro (4240)

Alan Gabriel Martins Silva (4663)

¹Instituto de Ciências Exatas e Tecnológicas,
Universidade Federal de Viçosa, Florestal, MG, Brasil

1. Introdução

Este trabalho apresenta a quarta etapa do desenvolvimento do projeto de sistemas distribuídos proposto na disciplina CCF355, com o objetivo de desenvolver um sistema baseado no conceito do jogo Super Trufo, onde os usuários podem colecionar e jogar com cartas que representam personagens, lugares ou objetos.

Essa etapa finaliza a implementação do projeto, com a modificação de SOCKETS para PYRO5 com Nameserver.

2. Mudanças Propostas

A principal modificação proposta por essa etapa é a remoção da API Sockets, e a implementação do Middleware PYRO5. Essas mudanças foram feitas, com implementações de comunicações uni e bi-laterais.

```
[*] Starting NameServer...

Not starting broadcast server for IPv6.
NS running on localhost:25555 (:::1)
URI = PYRO:Pyro.NameServer@localhost:25555

[*] Starting server...
[*] Loading StreamingAssets...
[*] StreamingAssets loaded.
[*] Initializing database...
[*] Server started.
[*] Log Level: 4
[*] Server terminal started.
[*] Matchmaking Server Started.
[*] Initialization Done.
>> Type 'shutdown' to stop the server.
>> Type 'help' for available commands.
```

3. Servidor

No servidor, algumas mudanças foram feitas ambas na parte de partidas quanto na parte interna principal.

Ao iniciar o servidor agora, o Pyro5 Name Server é iniciado com a porta de escolha, a do nosso programa, a porta 25555. A seguir, ao invés de iniciarmos com Sockets a conexão, iniciaremos com o Pyro5, localizando o name_server recém inicializado e registrando a classe `ufv_wilds` como `"ufv_wilds.server"`

Teremos também uma nova thread de heartbeat, que será responsável por verificar a conectividade dos usuários caso caiam da conexão.

3.1. Classe UFVWildsServer

Esse é o objeto responsável por servir de objeto PYRO para o cliente comunicar, esse objeto funciona com uma comunicação prioritariamente uni-lateral, sem se conectar ao objeto cliente, sendo utilizada apenas para receber chamadas em sua grande maioria, com exceção de uma função.

As funções dessa classe são:

- **set_connection:** Retorna um `client_id` unico
- **heartbeats_check:** verifica se alguém caiu da conexão a mais de um minuto.
- **update_heartbeats:** Faz a atualização do heartbeat de um cliente
- **register:** Registra um usuario
- **login:** Loga um usuario, retorna um token
- **logoff:** Remove todas as informações de um usuario
- **check_cards:** Retorna a lista de cartas de um usuario
- **check_card:** Verifica as informações de uma carta unica.
- **check_decks:** Retorna uma lista de decks
- **check_deck:** Verifica as informações de um deck específico
- **activate_deck:** Ativa um deck como principal
- **create_deck:** Cria um deck
- **delete_deck:** Deleta um deck
- **add_card_to_deck:** Adiciona uma carta a um deck
- **remove_card_from_deck:** Remove uma carta de um deck
- **match_search:** Unica função que chama o objeto cliente, faz as configurações necessarias e coloca o usuário em uma lista de busca de partida juntos ao seu Proxy. Chama o objeto cliente para anunciar isso
- **get_file_list:** Retorna uma lista de objetos
- **request_file:** Retorna um objeto (Imagem) pela rede.

3.2. Heartbeat Thread

Apenas faz a chamada a cada 5 segundos da função de checar batimentos do servidor.

3.3. Match

A partida também sofreu modificações consideráveis, agora, ao invés de sockets, tudo é controlado por PYRO5, e, quando uma partida começa, o servidor faz chamadas a interface dos jogadores da partida na thread `play_field`, obtido por um PROXY do Pyro5 ao se pesquisar uma partida. durante a partida, ao invés do jogador enviar para o servidor seu próximo passo, o servidor envia ao cliente aonde na partida o cliente está, e recebe como retorno a decisão feita pelo jogador.

4. Cliente

No cliente, algumas modificações importantes também foram feitas. A primeira, e que, ao se iniciar o jogo, o nome server é localizado, o servidor `ufv_wilds.server` é procurado e conectado, e uma conexão é estabelecida, com um `client_id` sendo retornado. após isso, a classe exposta do cliente `Client()` é registrada, com o nome **"Client-Match client_id"**. Essa decisão foi tomada visto que, se cada cliente registrasse com o mesmo nome, o name-server simplesmente sobrescreveria o ultimo registro. Dessa forma, como cada `client_id` é unico, cada usuario tem um endereçamento UNICO no nameserver também.

A seguir, os arquivos existentes no servidor na pasta "StreamingAssets" são verificados, e, caso falte arquivos no usuario, são transferidos para o usuario.

Uma thread de heartbeat é criada, na qual ira chamar a função `update_heartbeat` do servidor, a thread de interface grafica é inicializada com Pygame, e a thread de loop de requisições do PYRO é criada também.

4.1. Envio de comandos [Fora de partida]

: O sistema para enviar comandos ainda é o mesmo, porém, sem a thread de recebimento de mensagens. Agora, a Interface envia uma mensagem ao `Message_queue`, e essa é enviada a função `package_message` como antes. A diferença, e que essa função ira chamar os objetos do servidor ao invés de simplesmente enviar um JSON. O retorno dessas chamadas é colocado no `response_queue`, e a Interface gráfica então reage da forma esperada;

4.2. Envio de comandos [Dentro da partida]

: Aqui, o controle de fluxo vai para o servidor, com ele chamando o objeto `Cliente()`. Esse objeto ira funcionar de forma parecida ao envio de comandos fora de partida, com `Message_queue` e `Response_Queue` fazendo o mesmo trabalho, porém, agora, o cliente faz as requisições.

Essa classe é basicamente o resto da thread "Receive_Message" da etapa de sockets, porém, agora transformada como um Objeto Pyro apenas para as funções que necessitam de sincronização e comunicação dupla de forma extrema.

- **match_search**: É chamado para avisar a busca
- **match_start**: Recebe quais jogadores estão na partida
- **match_end**: Finaliza a partida e retorna o controle ao cliente
- **select_attribute**: Requisita um atributo do cliente
- **select_card**: Requisita uma carta do cliente.
- **your_turn**: Avisa que é o turno do cliente.
- **opponent_turn**: Avisa que é o turno do oponente.
- **turn_end**: Finaliza o turno.
- **card_hand**: Entrega ao cliente a sua mão.
- **reward**: Mostra qual carta foi recebida pelo cliente.

5. Classes

```
@Pyro5.api.expose
class UFWildsServer:
    def __init__(self): ...

    #Connection Management
    def set_connection(self):
        #make unique ID
        client_id = hashlib.md5(str(datetime.datetime.now()).encode()).hexdigest()
        self.clients.append(client_id)
        return client_id
    def heartbeats_check(self): ...
    def update_heartbeat(self, client_id): ...

    #Account Management

    def register(self, username, password): ...
    def login(self, username, password, client_id): ...
    def logoff(self, client_id): ...

    # Card Management
    def check_cards(self, token): ...
    def check_card(self, token, card_name): ...
    def check_decks(self, token): ...
    def check_deck(self, token, deck_name): ...
    def activate_deck(self, token, deck_name): ...
    def create_deck(self, token, deck_name): ...
    def delete_deck(self, token, deck_name): ...
    def add_card_to_deck(self, token, deck_name, card_name): ...
    def remove_card_from_deck(self, token, deck_name, card_name): ...

    #match search

    def match_search(self, token, client_id, URI): ...

    #Download Management
    def get_file_list(self, client_id): ...
    def request_file(self, client_id, image_path): ...
```

```
@Pyro5.api.expose
class Client:
    def __init__(self): ...

    def match_search(self): ...

    def match_start(self, player_1, player_2, player_3): ...
    def match_end(self): ...
    def select_attribute(self): ...
    def select_card(self, attribute): ...
    def your_turn(self): ...
    def opponent_turn(self): ...
    def turn_end(self, winner): ...
    def card_hand(self, cards, hand): ...
    def reward(self, reward): ...
    def error(self, message): ...
```

6. Observações sobre a Implementação

Ao final dessa entrega, a implementação do jogo UFV_WILDS se torna concluída com PYRO5. Essa etapa foi mais fácil que a anterior, vista que o jogo já estava basicamente pronto, e que, graças ao código extremamente modular desenvolvido na etapa 3, estava basicamente separado entre função / sockets de forma extrema. Essa etapa foi simplesmente remover o socket e colocar o Pyro no lugar.

A parte mais complexa foi a lógica de partida, que teve que ser retrabalhada um pouco por necessitar muito de socket, mas, como dito acima, cada parte estava já modularizada.

Existem três contas pre-registradas no banco de dados da entrega: mixxs, marcus e alan, todas com a senha 123456.